

Assignment 4 — Mental Checklist

Planning & Reasoning with Language Models · Session 5

A practical checklist of concepts and pitfalls to keep in mind while building your assignment. Organized in the order you'll actually do the work, from project setup through submission.

Before You Start — Understanding the Brief

What you're building: A creative, multi-step application that uses the LLM Gateway, Pydantic, and a validated prompt structure.

What's forbidden:

- Simple summarizers ("summarize this webpage")
- Stock or crypto analyzers
- Generic data analysis tools
- Anything that's a single LLM call

What's required:

- Multiple steps with 4–7+ LLM calls
- Multi-step reasoning visible in the output
- Use of LLM Gateway (V2 ideally) — you must not call a single LLM directly
- Pydantic for all input/output structures
- A prompt that passes the prompt-evaluator markdown test
- README documenting the prompt qualifiers and tests
- YouTube demo video of the working tool

Mindset for picking the topic: Think creative. "Analyze the election" is better than "analyze this stock." "Predict the next chief minister" is better than "summarize this article." The application should genuinely require the model to enter reasoning mode and do multi-step work.

Step 1 — Set Up Your Project with UV

- Run `uv init my_project` to scaffold the project.
- Add dependencies with `uv add pydantic`, `uv add httpx`, etc. — never edit `pyproject.toml` by hand.
- Tell Claude/Cursor explicitly: **"use UV for this project"** — it defaults to pip otherwise.
- Don't commit your `.venv` folder; share `pyproject.toml` and `.python-version` instead.

Keep in mind: Anyone running `uv run main.py` on your repo will get the dependencies installed automatically. This matters for the grader.

Step 2 — Wire Up the LLM Gateway

- Download the Gateway V2 code from the LMS.
- Create your `.env` file with API keys for all 7 providers: **Gemini, Groq, Cerebras, Nvidia, OpenRouter, GitHub Models, Ollama**.
- Start the gateway with `run . sh`. It runs on port 8100 for V2.
- Test it via the dashboard before integrating — confirm each provider works.

Keep in mind:

- You must use **multiple LLMs** in your assignment — don't hardcode one provider.
- Don't crash on 503s. Error handling belongs in the gateway, not in your agent.
- Be clever: use a fast model (Cerebras) for simple decisions and a stronger model (Gemini or GitHub's GPT-4.1) for complex orchestration.
- Caching only helps if your app repeats system prompts — useful but not required.

Step 3 — Design Your Pydantic Schemas First

Before writing any LLM call, define your data structures.

Every LLM output schema should have:

- A reasoning field — gives the model a place to think inside structured JSON. **Without this, the model jumps straight to the answer.**
- A confidence field (0–1 or low/medium/high) — forces the model to evaluate whether the answer matches the query.
- A `tool_call` or `final_answer` field — never both expected at once.
- Optional fields for anything the LLM might forget (use `Optional[...]`) — reduces failure rates.

For tool input schemas:

- Use `Field(description="...")` for every field — this description goes straight into the LLM's tool definition.
- Add constraints (`le=7`, `ge=0`, regex patterns) — Pydantic validates these automatically.

Keep in mind:

- `BaseModel`, not `dataclass`. `Dataclass` has no JSON schema or coercion.
- Pydantic tries coercion before failing — if the model returns "5" instead of 5, you don't need to handle it.
- When validation fails, Pydantic returns *why* — feed the error back to the LLM rather than blindly retrying.

Step 4 — Design the Prompt (Your Actual Job)

Claude can write the orchestration code. Claude cannot design good prompts for you. This is where you earn your keep.

Your system prompt must include:

- **Role definition** — who is this agent?
- **Explicit reasoning instructions** — "think step by step" or a reasoning field requirement
- **Output format** — reference the Pydantic schema
- **Tool usage rules** — when to call tools, when not to
- **Self-check requirement** — "Is your result reasonable? Does it answer the original query?"
- **Confidence requirement** — emit a confidence score with your answer
- **Fallback behavior** — what to do if a tool fails or the model is uncertain
- **Reasoning-type tag** — was this arithmetic? logic? entity recall?

Keep in mind:

- Keep system prompts **short** in V2. Tools go in a separate field, not in the prompt.
- Few-shot examples (showing one good reasoning chain) help with weaker models like Groq's Llama.
- Don't force JSON output verbally — use the V2 `response_format` with a Pydantic schema.

Step 5 — Validate Your Prompt with the Evaluator

This is a **hard requirement** of the assignment.

- Download the prompt-evaluator markdown from the LMS.
- Paste it into Claude / ChatGPT / Cursor.
- Paste your application prompt below it.
- The evaluator scores you on: explicit reasoning, structured output format, separation of reasoning and tools, conversation loop support, instructional formatting, internal self-checks, reasoning-time awareness, error/fallback handling.
- Iterate until your prompt passes all checks.

Keep in mind:

- Even the instructor's prompt failed on internal self-checks, reasoning-type awareness, and fallbacks. Don't be surprised if yours fails several criteria first.
- Save the evaluator's output — you'll include it in your README as proof.
- Eventually, build this validation step into a reusable service of your own.

Step 6 — Build the Agent Architecture (V5 Foundation)

Use Agent V5 patterns:

- Native tool use, no manual JSON parser
- Canonical tool calls returned by the gateway
- Pydantic AgentTrace object for history — not a list of dicts
- `asyncio.TaskGroup` for parallel tool calls when the LLM requests independent operations
- Verifier agent at the end to double-check the final answer

Trace every interaction:

For each LLM call and tool call, log `provider`, `model`, `latency`, `input_tokens`, `output_tokens`, `cache_hit`, `tool_name`, `tool_args`, `text_payload`. The grader will want to see telemetry.

Keep in mind:

- Agent V5 is a "weak foundation" — you'll need to extend it.
- Tool definitions go **separately** from the system prompt in V2. The router falls back automatically for older models.
- MCP tool outputs are **always strings** unless it's an image/file object.

Step 7 — Think About Tool Categories

Simple tools (you provide): add, subtract, query a known API endpoint.

Complex reusable tools (still you-provided, but heavyweight): web search, PDF parsing, headless-browser scraping. These need careful error handling — websites block bots, PDFs have images and tables.

Eventually (session 10+ topic): tools the agent writes for itself via a coding sub-agent. Don't try this in Assignment 4.

Watch out: Don't pick "scrape NSE for stock prices" — the instructor specifically warned NSE's site is unparseable. If your tool can fail at runtime (web search, API rate limits), build retry logic and tell the LLM about failures via Pydantic.

Step 8 — Multi-Step Reasoning Must Be Visible

The grader needs to see the model actually entering reasoning mode and making multiple LLM calls. Plan your application so there are clear stages:

- Initial query parsing and decomposition
- Tool selection and parallel/sequential dispatch
- Intermediate validation checkpoints

- Final answer construction
- Verification pass

Keep in mind:

- 4–5 LLM calls is the floor. Aim higher (10–15 is fine with the gateway).
- If you can structure two independent calls in parallel, use `asyncio.TaskGroup` to demonstrate this V5 feature.
- The "reasoning" field at each stage should show genuinely different reasoning, not boilerplate.

Step 9 — Document and Demo

README must contain:

- What the application does and why it's creative/non-trivial
- The prompt itself
- The prompt-evaluator output (showing it passes the criteria)
- Setup instructions (UV-based)
- Architecture diagram or description (which agents/tools call what)
- Sample input/output traces showing the multi-step reasoning
- Link to the YouTube demo

YouTube video should show:

- The tool running end-to-end
- The terminal/log output showing multiple LLM calls and tool dispatch
- The gateway dashboard showing which providers were used
- The Pydantic-validated final output

Submit on the new LMS (Google login only) once enrollment emails arrive. Submit Assignment 4 only — old assignments stay on Canvas.

Common Pitfalls to Avoid

#	Pitfall
1	Forgetting the reasoning field in Pydantic output — model can't think and jumps to answers.
2	Hardcoding a single LLM — the assignment requires multiple LLMs via gateway.
3	Using LangChain, LangGraph, or Google ADK — explicitly discouraged. They abstract orchestration away.
4	Manual JSON parsing with regex — that's V4 thinking. Use Pydantic's <code>model_validate_json()</code> in V5.
5	Skipping the prompt evaluator — your prompt must visibly pass the checks.
6	Picking a forbidden topic — no summarizers, no stock/crypto, no generic analyzers.
7	One LLM call disguised as many — calls must genuinely require reasoning at each step.
8	Forgetting telemetry/tracing — the trace output is the evidence your agent did real work.

9	Assuming caching works on free tiers — it won't (except locally with Ollama).
10	Switching LLMs mid-conversation without realizing the cost — every switch re-sends history.

Quick Sanity Check Before Submitting

Before you call it done, ask yourself:

- Does my prompt pass the evaluator markdown?
- Does my Pydantic schema include reasoning and confidence fields?
- Am I using V2 gateway features (separate tool calling, native tool use)?
- Do I have at least 4–7 distinct LLM calls with genuine reasoning at each?
- Is my topic creative and forbidden-list-compliant?
- Does my README include the evaluator output as proof?
- Can someone clone the repo and run `uv run main.py` without manual setup?
- Is the YouTube video clear about what's happening at each step?

If all of these are **yes**, you're in good shape.

Based on Session 5 transcript: Planning and Reasoning with Language Models.